Two-Dimensional Poisson Equation on the Unit Square

Jacobi, Gauss–Seidel and Successive Under-Relaxation Solvers in C: Accuracy, Convergence and Stopping Criteria

Course Project 4 – OE5450 Numerical Techniques in Ocean Hydrodynamics (Assignment 3, Jan–May 2025)

Karthick J (OE23S018)
Department of Ocean Engineering, IIT Madras
Faculty: Prof. K. Murali

Revised report, October 2026

Abstract

The Poisson equation $\nabla^2 T = Q$ with a two-node source/sink pair is solved on the unit square for two sets of boundary conditions (Dirichlet on all walls; and Dirichlet on three walls with a prescribed flux $\partial T/\partial x = -1$ on the fourth) and three grids ($\Delta = 0.1, 0.05, 0.01$) with point Jacobi, Gauss–Seidel and successive under-relaxation (SUR, $\alpha = 0.7$). Review of the submitted C codes found an out-of-bounds write in the Jacobi boundary routine, boundary conditions rotated by 90° relative to the problem statement, a floating-point truncation that moved one source to $x = 0.44$, per-iteration output of the whole field (up to 0.7 GB per run), and no implementation of Case 2. After correction, all solvers converge to the direct sparse solution of the discrete system, which agrees with the analytical Laplace solution with second-order accuracy (3.3×10^{-4} interior error at $\Delta = 0.01$). The study shows that the absolute increment test $\max|T^{k+1} - T^k| < 10^{-4}$ leaves an iteration error of order $\epsilon\rho/(1-\rho)$, i.e. 0.10–0.21 on the finest grid, more than half of the solution magnitude in Case 2; a tolerance of 10^{-6} reduces it to 10^{-3}. Iteration counts grow as $N^2 = \Delta^{-2}$, as predicted by $\rho_J = \cos\pi\Delta$.

Contents

1 Problem definition **2**
1.1 Analytical solution of the source-free problem . . . 2

2 Numerical method **2**
2.1 Discretisation . . . 2
2.2 Iterative schemes . . . 2
2.3 Stopping criterion and iteration error . . . 3
2.4 Algorithm . . . 3

3 Code review and corrections **3**

4 Results **4**
4.1 Temperature fields . . . 4
4.2 Effect of the sources . . . 5
4.3 Accuracy of the discretisation . . . 5
4.4 Convergence and the stopping criterion . . . 6

5 Conclusions **7**

A Corrected source: Gauss–Seidel **7**

1 Problem definition

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = Q(x,y), \qquad 0 \le x, y \le 1, \tag{1}$$

with nodal sources $Q = +1$ at $(0.45, 0.5)$ and $Q = -1$ at $(0.55, 0.5)$ (zero elsewhere).

	Case 1	Case 2
$y = 0$, $y = 1$	$T = 0$	$T = 0$
$x = 0$	$T = 10$	$T = 0$
$x = 1$	$T = 10$	$\partial T/\partial x = -1$

Table 1: Boundary conditions (from the problem statement).

Grid spacings $\Delta = \Delta x = \Delta y = 0.1, 0.05, 0.01$ ($N = 1/\Delta = 10, 20, 100$) were to be compared, using Jacobi, Gauss–Seidel and successive under-relaxation iterations with the stopping criterion $\max |T_{ij}^{k+1} - T_{ij}^{k}| < 10^{-4}$.

1.1 Analytical solution of the source-free problem

Because Q is non-zero at only two nodes, its effect on T is very small (Section 4.2), and the analytical Laplace solutions provide the accuracy reference:

$$\text{Case 1:} \quad T = \sum_{n \text{ odd}} \frac{40}{n\pi} \sin(n\pi y) \, \frac{\cosh\big(n\pi(x - \frac{1}{2})\big)}{\cosh(n\pi/2)}, \tag{2}$$

$$\text{Case 2:} \quad T = -\sum_{n \text{ odd}} \frac{4}{n^2\pi^2} \sin(n\pi y) \, \frac{\sinh(n\pi x)}{\cosh(n\pi)}, \tag{3}$$

the second following from expanding the boundary flux $-1 = \sum b_n \sin n\pi y$, $b_n = -4/(n\pi)$ for odd n. Case 1 has temperature discontinuities at the four corners, which limit pointwise accuracy there; errors are therefore measured on $[0.2, 0.8]^2$.

2 Numerical method

2.1 Discretisation

The five-point central approximation of (1) on a uniform mesh is

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = \Delta^2 Q_{i,j}, \qquad \tau = \frac{\Delta^2}{12}\big(T_{xxxx} + T_{yyyy}\big) + \mathcal{O}(\Delta^4). \tag{4}$$

The Case 2 flux condition is closed with the second-order one-sided formula

$$\frac{3T_{N,j} - 4T_{N-1,j} + T_{N-2,j}}{2\Delta} = -1 \quad \Longrightarrow \quad T_{N,j} = \tfrac{1}{3}\big(4T_{N-1,j} - T_{N-2,j} - 2\Delta\big), \tag{5}$$

applied after every sweep.

2.2 Iterative schemes

Writing (4) as $A\mathbf{T} = \mathbf{b}$, $A = D - L - U$:

$$\text{Jacobi:} \quad T_{ij}^{k+1} = \tfrac{1}{4}\big(T_{i+1,j}^{k} + T_{i-1,j}^{k} + T_{i,j+1}^{k} + T_{i,j-1}^{k} - \Delta^2 Q_{ij}\big), \qquad \rho_J = \cos\pi\Delta, \tag{6}$$

$$\text{Gauss–Seidel:} \quad T_{ij}^{k+1} = \tfrac{1}{4}\big(T_{i-1,j}^{k+1} + T_{i+1,j}^{k} + T_{i,j-1}^{k+1} + T_{i,j+1}^{k} - \Delta^2 Q_{ij}\big), \qquad \rho_{GS} = \rho_J^2, \tag{7}$$

$$\text{SUR:} \quad T_{ij}^{k+1} = T_{ij}^{k} + \alpha\big(\tilde{T}_{ij}^{GS} - T_{ij}^{k}\big), \ \alpha = 0.7, \qquad \rho_{SUR} \approx 1 - \alpha(1 - \rho_{GS}). \tag{8}$$

For $\Delta = 0.1, 0.05, 0.01$: $\rho_J = 0.9511, 0.9877, 0.99951$ and $\rho_{GS} = 0.9045, 0.9755, 0.99901$. The number of iterations to reduce the error by a fixed factor is $\propto 1/(1-\rho) \approx 2/(\pi^2\Delta^2)$ for Jacobi, i.e. it grows as N^2. Gauss–Seidel needs half as many, and under-relaxation ($\alpha < 1$) makes SUR *slower* than Gauss–Seidel by about $1/\alpha$. Under-relaxation is useful for non-linear or stiff problems, not for the linear Poisson equation, for which over-relaxation ($\omega \approx 2/(1+\sin\pi\Delta)$) is optimal.

2.3 Stopping criterion and iteration error

For a linear stationary iteration with spectral radius ρ, the error $e^k = T^k - T$ and the increment $\delta^k = T^{k+1} - T^k$ are related asymptotically by

$$\|e^{k+1}\| \approx \frac{\rho}{1-\rho}\,\|\delta^k\|. \tag{9}$$

With $\|\delta\| < \epsilon = 10^{-4}$ and $\Delta = 0.01$ this gives $e \approx 0.20$ for Jacobi and 0.10 for Gauss–Seidel: the increment test is far looser than it appears on fine grids. This is quantified in Section 4.4.

2.4 Algorithm

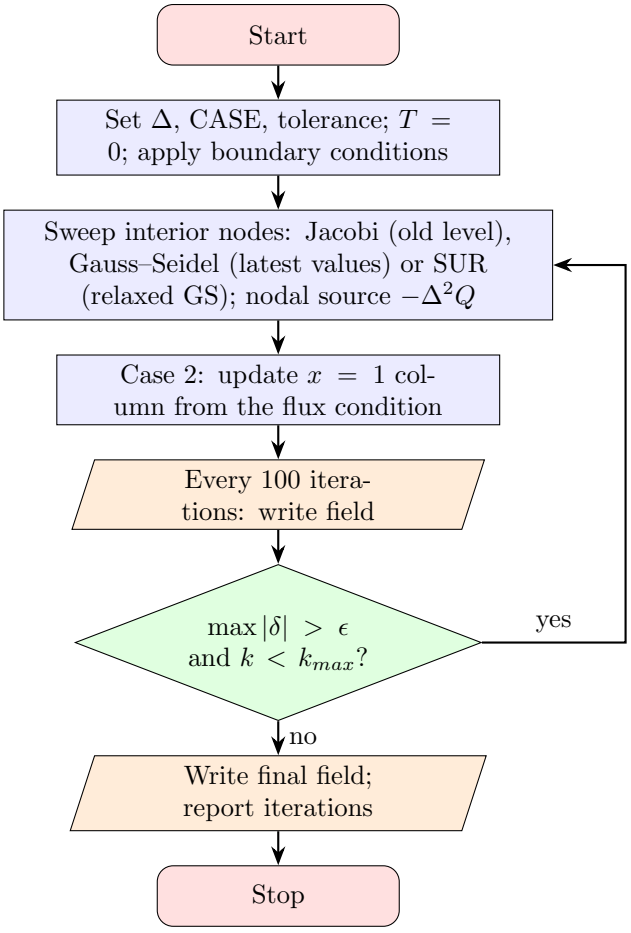


Figure 1: Common structure of `code/jacobi.c`, `gauss_seidel.c` and `sur.c`.

3 Code review and corrections

The three codes were taken from the submitted report (`original/code_from_report/`). Run unchanged on the three grids, the Gauss–Seidel and SUR codes reproduce the Case 1 iteration counts of the original table exactly (91/308/4441 and 165/553/7753), and so does Jacobi for $\Delta = 0.05$ and

0.01 (559, 7491). For $\Delta = 0.1$ the Jacobi code gives 166 iterations, which is the Case 2 entry of the table, so the two Jacobi entries at $\Delta = 0.1$ appear to be interchanged. The following defects were corrected:

Location	Problem	Correction
Jacobi, `setBoundaryConditions`	`T[MX-1][NY] = T_min` writes one element beyond the end of `T` (undefined behaviour) and the right wall is never set.	Index `[MX-1][j]`.
all three, boundary conditions	T_{max} was applied on $y = 0, 1$ and T_{min} on $x = 0, 1$, i.e. the problem was rotated by 90° relative to the statement (Case 1: $T = 10$ on $x = 0, 1$, $T = 0$ on $y = 0, 1$). For a square with these symmetric conditions the field is merely rotated, but it is inconsistent with the stated problem and with the source positions.	Boundary values applied as stated.
all three, source location	`(int)(0.45/delta_x)` truncates 44.999 … to 44 for $\Delta = 0.01$: the source sat at $x = 0.44$, the sink at 0.55.	Rounding to the nearest node.
all three, output	The whole field was written every iteration ($\approx$ 0.7 GB for $\Delta = 0.01$; the 2.6 GB files in the original folders have the same cause).	Written every 100 iterations and at the end.
Case 2	Not present in the submitted listing although Case 2 results are tabulated.	Added with a `CASE` switch and a second-order flux closure.

Table 2: Defects and corrections (marked `[FIX]`/`[ADDED]` in `code/`).

Not changed: the nodal interpretation of Q, the iteration formulas, the relaxation factor, the tolerance and the use of `float` in the Jacobi code (adequate for $\epsilon = 10^{-4}$, but it limits the attainable tolerance to $\sim 10^{-6}$ for $T \sim 10$). The value 12 345 in the original Case 2 table exceeds the iteration cap (`Max_iteration` = 10 000) of the listed codes and could not be reproduced.

4 Results

4.1 Temperature fields

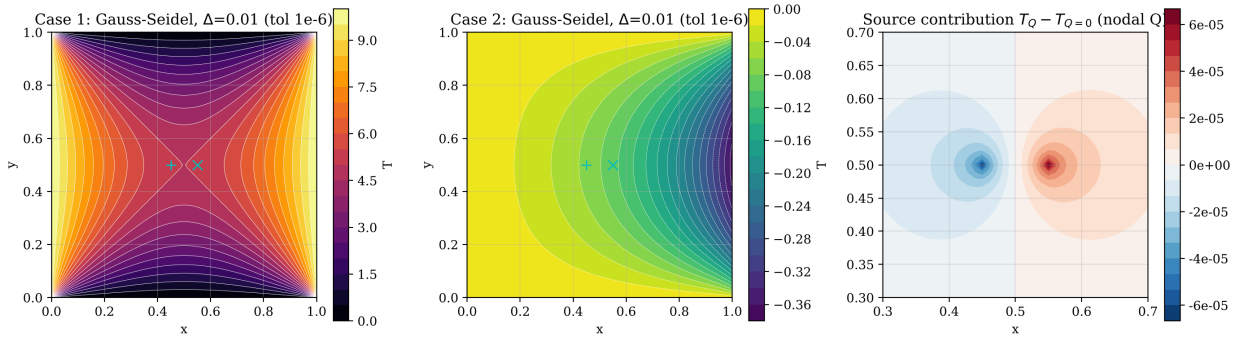


Figure 2: Converged fields ($\Delta = 0.01$, Gauss–Seidel, tolerance 10^{-6}) for Cases 1 and 2, and the contribution of the nodal sources (+: source, ×: sink).

Case 1 is symmetric about both centre lines with a saddle at the centre ($T = 5$ exactly, by symmetry). In Case 2 the outward flux through $x = 1$ draws the temperature negative; the minimum, $T(1, 0.5) =$

-0.37, lies at the middle of the flux boundary.

4.2 Effect of the sources

With Q defined as a nodal value, the source and sink together perturb the field by at most 6.2×10^{-5} (right panel above): the source strength integrated over one cell is $Q\Delta^2$, which vanishes as the grid is refined. If a unit *point* source were intended ($\int Q\, dA = \pm 1$), the nodal value must scale as $Q = \pm 1/\Delta^2$; this case (not part of the original assignment, shown as an extension in `results/fig5_extension_point_dipole.png`) produces a dipole of amplitude 0.62. The problem statement should specify which definition is meant.

4.3 Accuracy of the discretisation

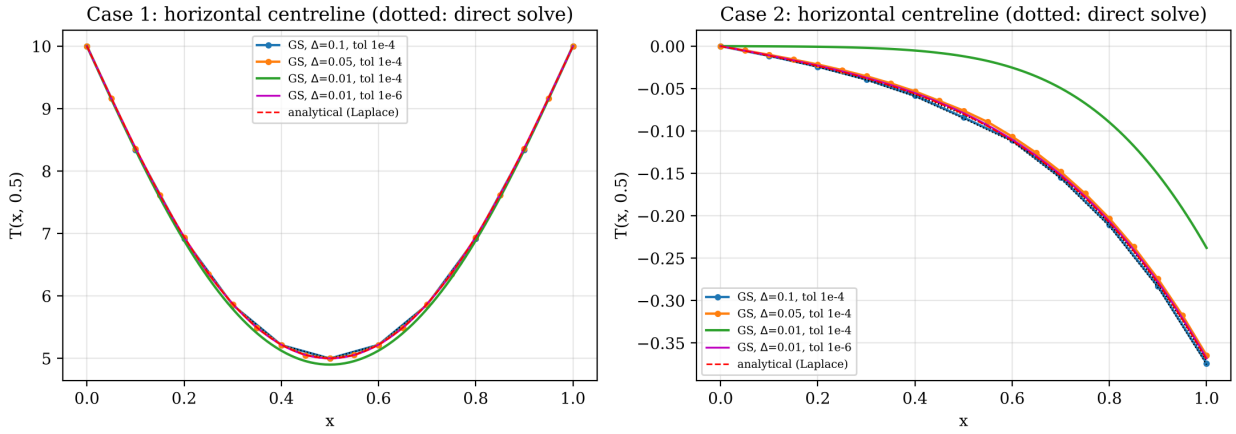


Figure 3: $T(x, 0.5)$: Gauss–Seidel results at both tolerances, direct solution of the discrete system (dotted) and analytical Laplace solution (dashed).

Δ	Case 1: $\max \lvert T_h - T \rvert$ on $[0.2, 0.8]^2$	Case 2: same	Case 2: $\lvert T_h - T \rvert$ at centre
0.1	2.97×10^{-2}	5.39×10^{-3}	5.26×10^{-3}
0.05	7.99×10^{-3}	1.56×10^{-3}	7.49×10^{-4}
0.01	3.30×10^{-4}	9.01×10^{-5}	3.20×10^{-5}

Table 3: Discretisation error of the exact discrete solution.

Halving Δ reduces the Case 1 error by 3.7 and the five-fold refinement from 0.05 to 0.01 by $24 \approx 5^2$, confirming second-order accuracy.

4.4 Convergence and the stopping criterion

Case	Δ		Jacobi iter.	Jacobi error	Gauss–Seidel iter.	Gauss–Seidel error	SUR ($\alpha = 0.7$) iter.	SUR ($\alpha = 0.7$) error
1	0.1	$\epsilon = 10^{-4}$	166	1.9×10^{-3}	91	9.1×10^{-4}	165	1.3×10^{-3}
1	0.05		559	7.9×10^{-3}	308	4.0×10^{-3}	553	5.2×10^{-3}
1	0.01		7 486	0.201	4 441	0.101	7 753	0.132
2	0.1		134	3.2×10^{-3}	84	1.6×10^{-3}	143	2.1×10^{-3}
2	0.05		360	1.3×10^{-2}	233	6.5×10^{-3}	393	8.5×10^{-3}
2	0.01		761	0.214	982	0.132	1 343	0.161
1	0.01	$\epsilon = 10^{-6}$	19 426	5.7×10^{-4}	9 106	1.0×10^{-3}	16 419	1.3×10^{-3}
2	0.01		13 601	3.1×10^{-3}	7 909	1.6×10^{-3}	13 854	2.1×10^{-3}

Table 4: Iterations and remaining iteration error $\max |T^k - T_{direct}|$.

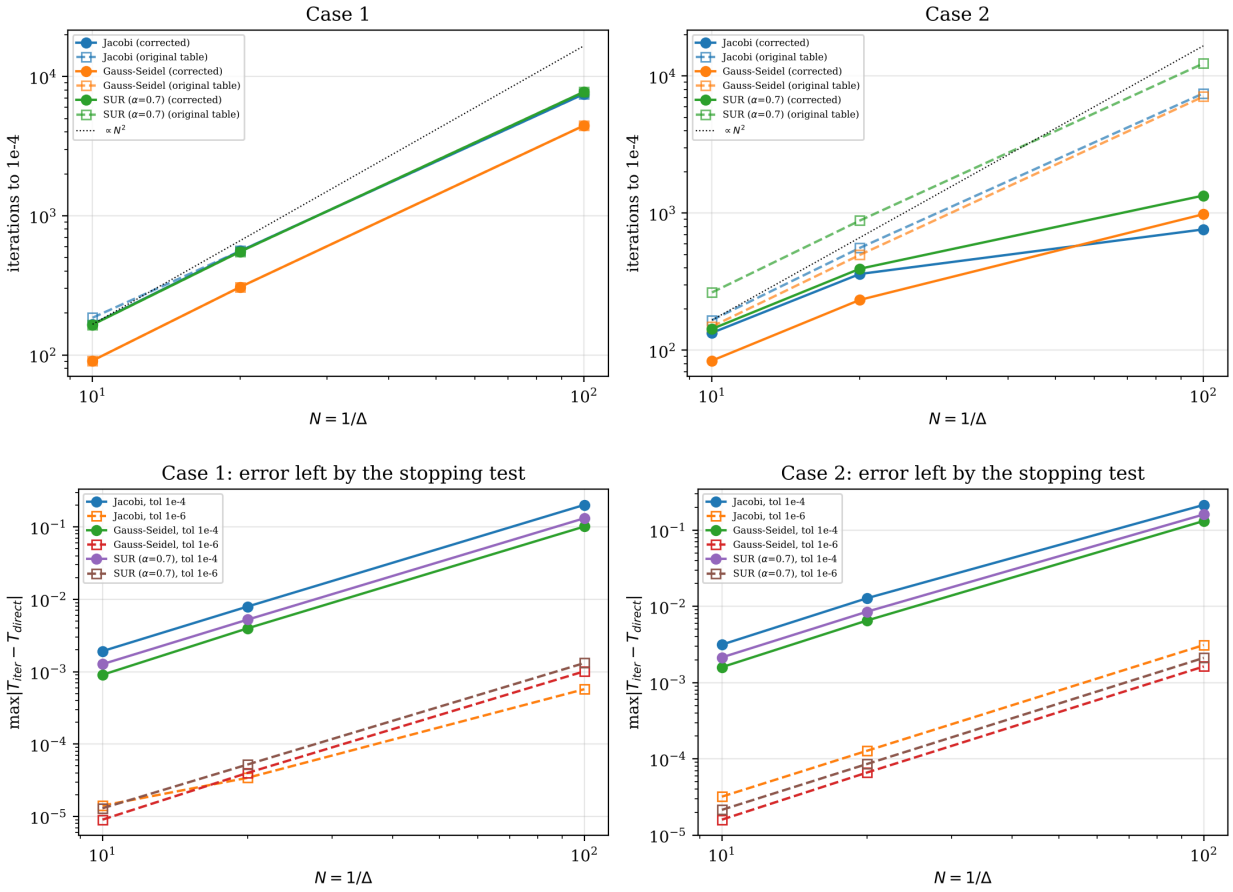


Figure 4: Top: iterations to $\epsilon = 10^{-4}$ against $N = 1/\Delta$ (corrected codes and the original table). Bottom: iteration error remaining at termination for both tolerances.

- **Case 1** follows the theory: Gauss–Seidel needs ≈ 0.55–0.6 of the Jacobi iterations, SUR with $\alpha = 0.7$ needs about as many as Jacobi, and the counts grow roughly as N^2.
- **The remaining error matches Eq. (9).** At $\Delta = 0.01$, $\epsilon\rho/(1-\rho) = 0.20$ (Jacobi) and 0.10 (Gauss–Seidel), against the measured 0.201 and 0.101.
- **Case 2 is under-converged at $\epsilon = 10^{-4}$.** Its solution is small ($|T| \leq 0.37$) and grows slowly from the zero initial guess, so the increments fall below 10^{-4} long before convergence

(761–1 343 iterations at $\Delta = 0.01$), leaving errors of more than half the solution magnitude. With $\epsilon = 10^{-6}$ the iteration counts rise to 7 900–13 900 and the error falls to $\leq 3 \times 10^{-3}$. The original Case 2 counts (7 081–12 345) are of this order, which suggests that the original Case 2 code used a different flux closure or tolerance.

5 Conclusions

- The corrected solvers converge to the direct solution of the discrete Poisson problem, which is second-order accurate (3.3×10^{-4} interior error at $\Delta = 0.01$ for Case 1).
- Iteration counts scale as Δ^{-2}; Gauss–Seidel is about twice as fast as Jacobi, and under-relaxation slows convergence for this linear problem.
- An absolute increment tolerance is not an error estimate: on the 101×101 grid $\epsilon = 10^{-4}$ leaves errors of 0.1–0.2, and in Case 2 more than 50% of the solution. A residual-based or relative criterion, $\epsilon \sim (1 - \rho)\times$ target accuracy, should be used.
- As defined (nodal $Q = \pm 1$), the source pair changes the solution by less than 10^{-4}; a unit point dipole would require $Q = \pm 1/\Delta^2$.

Reproducing the results

`bash run.sh` builds and runs all 18 combinations at $\epsilon = 10^{-4}$ and again at $\epsilon = 10^{-6}$ (only the constants are substituted), then calls `code/postprocess.py` (NumPy, SciPy, Matplotlib). Total run time is about 25 s.

A Corrected source: Gauss–Seidel

```
// 2-D Poisson equation, unit square, gauss seidel iteration.
// Original author: Karthick J (OE23S018), OE5450 Assignment 3 (code taken from the submitted report).
// Corrections are marked [FIX]; additions [ADDED].
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define L 1.0 // length of the domain (x and y)
#define delta_x 0.01 // length of the cell size in x axis
#define delta_y 0.01 // length of the cell size in y axis
#define MX ((int)(L / delta_x) + 1) // number of cells in x axis
#define NY ((int)(L / delta_y) + 1) // number of cells in y axis
#define ERROR 0.0001 // error for the convergence
#define T_max 10.0 // maximum temperature at the boundary condition
#define T_min 0.0 // minimum temperature at the boundary condition
#define Max_iteration 10000 // maximum iteration allowed for solving the equation
#define CASE 1 // [ADDED] boundary-condition case: 1 or 2 (see problem statement)
#define DTDX_RIGHT -1.0 // [ADDED] Case 2: prescribed dT/dx on x = 1
#define WRITE_EVERY 100 // [FIX] write the field every WRITE_EVERY iterations (was every iteration:
                        // up to ~0.7 GB per run for delta = 0.01)

double T[MX][NY]; // Temperature grid
double T_new[MX][NY]; // New temperature grid

// [FIX: orientation. The problem statement prescribes (Case 1) T = 10 on x = 0 and x = 1 and
// T = 0 on y = 0 and y = 1; the original code applied T_max on y = 0, 1 and T_min on x = 0, 1,
// i.e. the problem rotated by 90 degrees. (The Jacobi listing also wrote T[MX-1][NY], one
// element past the end of the array, so its right wall was never set.)]
// [ADDED: Case 2 -- T = 0 on y = 0, y = 1 and x = 0; dT/dx = -1 on x = 1 -- which was used for the
// Case 2 results in the original table but is not in the submitted listing.]
// Neumann condition, second-order one-sided: (3 T_N - 4 T_{N-1} + T_{N-2}) / (2 dx) = dT/dx
void applyNeumann() {
    for (int j = 1; j < NY - 1; j++) {
```

```
        T[MX - 1][j] = (4.0 * T[MX - 2][j] - T[MX - 3][j] + 2.0 * delta_x * DTDX_RIGHT) / 3.0;
    }
}

void setBoundaryConditions() {
    int i, j;
    // Bottom (y = 0) and top (y = 1) boundaries: T = T_min in both cases
    for (i = 0; i < MX; i++) {
        T[i][0] = T_min;
        T[i][NY - 1] = T_min;
    }
    // Left (x = 0) and right (x = 1) boundaries
    for (j = 0; j < NY; j++) {
        T[0][j] = (CASE == 1) ? T_max : T_min;
        if (CASE == 1) T[MX - 1][j] = T_max;
    }
    if (CASE == 2) applyNeumann();
}

// Function to perform Gauss-Seidel iteration
void gaussSeidelIteration(double *max_diff) {
    int i, j;
    double q1 = 1.0, q2 = -1.0;
    // [FIX: (int)(0.45/0.01) = 44 because 0.45/0.01 = 44.999...; round to the nearest node]
    int sx1 = (int)(0.45 / delta_x + 0.5), sy1 = (int)(0.5 / delta_y + 0.5);
    int sx2 = (int)(0.55 / delta_x + 0.5), sy2 = (int)(0.5 / delta_y + 0.5);

    *max_diff = 0.0; // Reset max_diff

    for (i = 1; i < MX - 1; i++) {
        for (j = 1; j < NY - 1; j++) {
            double source = 0;
            if (i == sx1 && j == sy1) source = q1;
            if (i == sx2 && j == sy2) source = q2;

            // Update T[i][j] using the most recent values
            double old_T = T[i][j];
            T_new[i][j] = 0.25 * (T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1] - (delta_x * delta_x) *
                 source);

            // Calculate error
            double error = fabs(T_new[i][j] - old_T);
            if (error > *max_diff) {
                *max_diff = error;
            }

            // Update T[i][j] immediately for Gauss-Seidel
            T[i][j] = T_new[i][j];
        }
    }
}

// Function to write grid to file
void writeGridToFile(FILE *file) {
    for (int i = 0; i < MX; i++) {
        for (int j = 0; j < NY; j++) {
            fprintf(file, "%f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
    fprintf(file, "\n\n"); // Two blank lines between datasets for gnuplot
}

int main() {
    double max_diff;
    int iter = 0;

    printf("Number of cells in the X-axis: %d\n", MX);
    printf("Number of cells in the Y-axis: %d\n", NY);

    // Initialize temperature array
    for (int i = 0; i < MX; i++) {
        for (int j = 0; j < NY; j++) {
            T[i][j] = 0.0; // Initialize all temperatures to 0
        }
```

```
    }

    // Set boundary conditions
    setBoundaryConditions();

    // Open output file
    FILE *outputFile = fopen("temperature_field.txt", "w");
    if (outputFile == NULL) {
        printf("Error opening output file!\n");
        return 1;
    }

    // Gauss-Seidel iteration
    do {
        gaussSeidelIteration(&max_diff);
        if (CASE == 2) applyNeumann(); // [ADDED] update the Neumann boundary after each sweep

        iter++;

        // Write grid to file every WRITE_EVERY iterations [FIX]
        if (iter % WRITE_EVERY == 0) writeGridToFile(outputFile);

    } while (max_diff > ERROR && iter < Max_iteration);

    // Write final state
    writeGridToFile(outputFile);

    // Close output file
    fclose(outputFile);

    if (iter >= Max_iteration) {
        printf("Failed to converge after %d iterations\n", Max_iteration);
    } else {
        printf("Converged after %d iterations\n", iter);
    }

    return 0;
}
```